nature communications

Article

https://doi.org/10.1038/s41467-026-70240-6

Bridging the latency gap with a continuous stream evaluation framework in event-driven perception

Received: 1 March 2025

Accepted: 23 February 2026

Published online: 16 March 2026

Check for updates

Jie Chu[1,2,6], Runze Zhang[1,6], Chu Yang[3], Zongyou Yu[1], Zongtao Bu[1], Haotian Liu[1], Florian Röhrbein[4], Alois Knoll[5], Guang Chen[1,2,5] ✉ & Changjun Jiang[1]

Neuromorphic vision systems process continuous event streams and offer transformative potential for real-time applications. However, their evaluation remains tethered to methodologies from RGB imaging. These approaches convert asynchronous event streams into synchronized frames and ignore perception latency, creating a critical gap between benchmarks and real-world performance. To address this, we introduce the STream-based lAtency-awaRe Evaluation (STARE) framework. STARE integrates two core components: Continuous Sampling, maximizing model throughput to reduce the impact of latency, and Latency-Aware Evaluation, quantifying latency-induced online accuracy. To rigorously validate STARE, we developed ESOT500, a high-dynamic object tracking dataset with 500 Hz annotations. Experiments reveal that latency severely degrades online accuracy by over 50%. We further introduce two model enhancement strategies: Asynchronous Tracking, a fast-slow architecture that boosts model throughput, and Context-Aware Sampling, which dynamically adapts input to handle low event density cases. Overall, our work bridges the latency gap between models' theoretical potential and real-world deployment.

Biological vision systems excel at perceiving dynamic environments through continuous, adaptive sensing[1–3]. In contrast, artificial vision systems, even those inspired by neural mechanisms, typically rely on discrete and frame-based processing inherited from conventional RGB cameras[4–8], as shown in Fig. 1a–b. This mismatch introduces a critical limitation: the temporal discontinuity of static frames inherently induces perceptual delay between sensory inputs[9–13]. Such delay accumulates in real-world applications, degrading performance in tasks requiring rapid reactions, such as autonomous navigation or human-robot interaction[9,14–17]. Event cameras, which are neuromorphic sensors that asynchronously encode pixel-level intensity changes at microsecond resolution, promise to bridge this gap by capturing continuous event streams[3,18–21].

However, the traditional perception framework, which forces continuous event stream into fixed-rate event frames[3,4,21–24], as shown in Fig. 1b, reintroduces perceptual delays and discards the sensor's inherent capacity for real-time, event-driven computation. Moreover, such framework typically evaluates model performance by comparing each model output against the ground truth of the corresponding input frame. This evaluation paradigm assumes instantaneous computation, failing to account for the influence of perception latency. We define perception latency as the total time elapsed from the moment

[1]Generalist Embodied Intelligence Lab, School of Computer Science and Technology, Tongji University, Shanghai, China. [2]Shanghai Innovation Institute, Shanghai, China. [3]Shanghai Westwell Technology Co., Ltd, Shanghai, China. [4]Chair of Neurorobotics, Technische Universität Chemnitz, Chemnitz, Germany. [5]Chair of Robotics, Artificial Intelligence and Real-time Systems, School of Computation, Information and Technology, Technische Universität München, München, Germany. [6]These authors contributed equally: Jie Chu, Runze Zhang. ✉e-mail: guangchen@tongji.edu.cn

https://doi.org/10.1038/s41467-026-70240-6

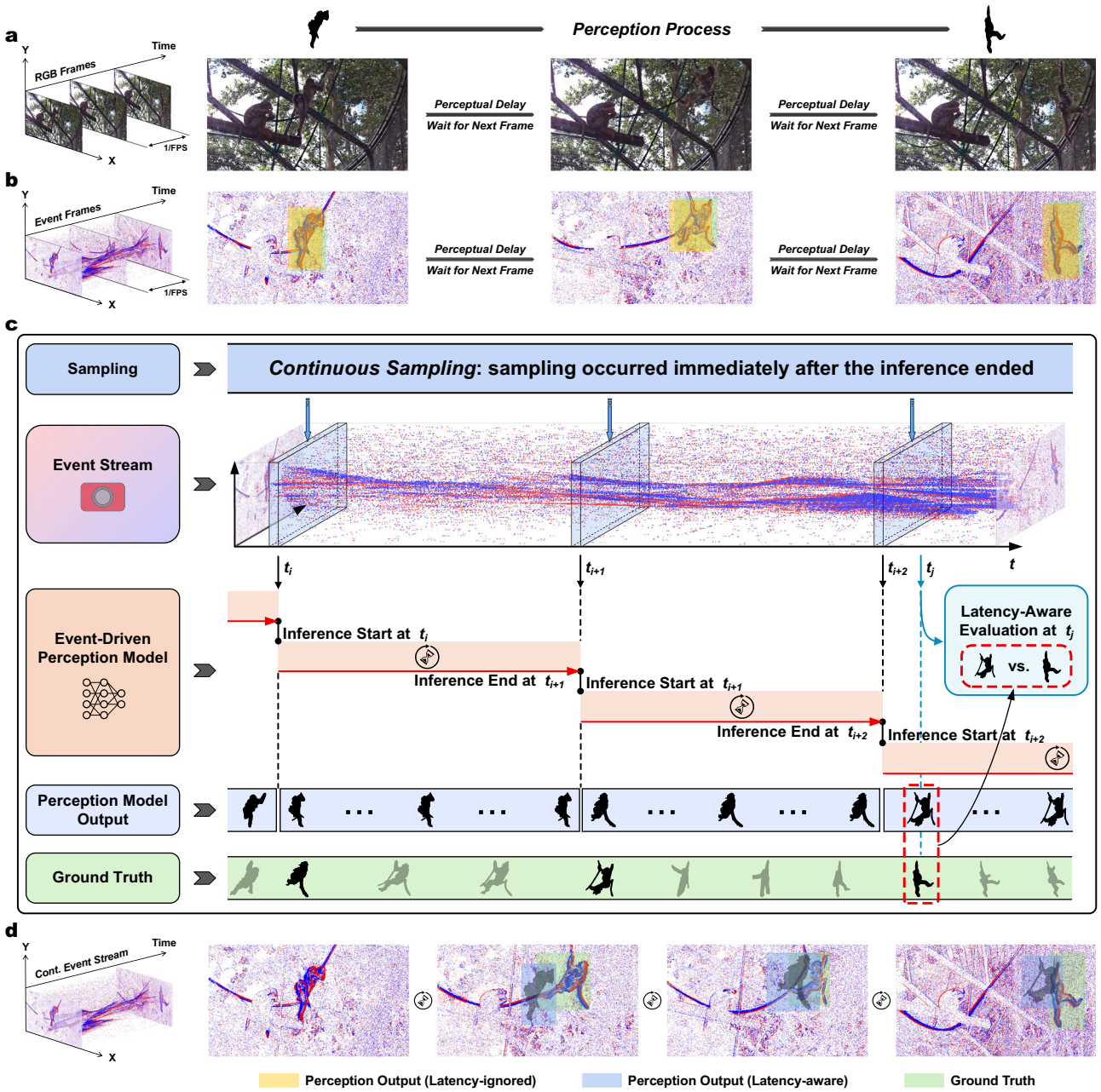
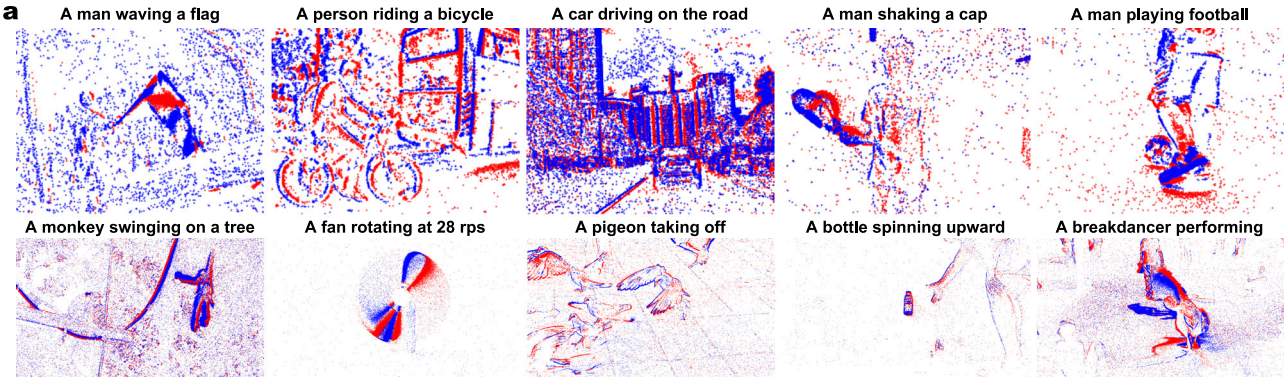


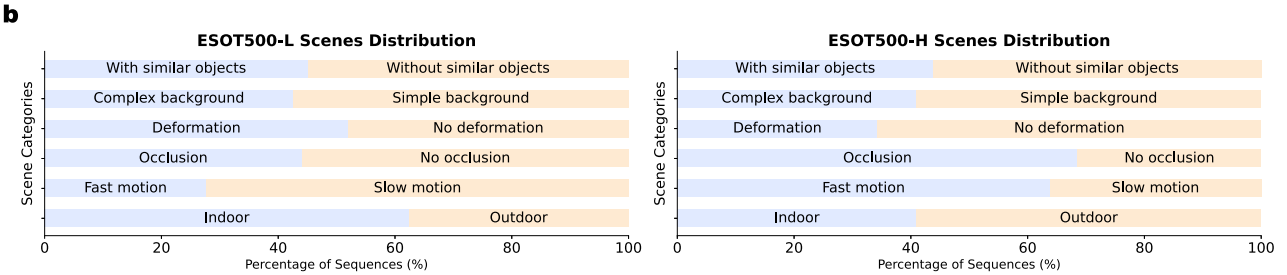
Fig. 1 | STream-based lAtency-awaRe Evaluation (STARE) for event-driven perception. a Traditional RGB perception pipeline. Visual information is captured as discrete, fixed-rate frames. The perception model processes frames sequentially, introducing perceptual delays as inference is gated by the arrival of each new frame (i.e., waiting for the next frame to start processing). **b** Frame-based, latency-ignored evaluation of event vision. Mirroring RGB paradigms, the continuous event stream is preprocessed into fixed-rate event frames. Model outputs (orange bounding box) are evaluated against ground truth within the corresponding input frame, ignoring the impact of perception latency on real-time accuracy. **c** The proposed STream-based lAtency-awaRe Evaluation (STARE) framework. STARE operates directly on the continuous event stream: inference initiates immediately after the prior cycle concludes (at timestamps t_i, t_{i+1}, t_{i+2}). Latency-Aware Evaluation matches each high-frequency ground truth (e.g., at t_j) to the latest model prediction, directly penalizing stale outputs. **d** Illustrative example of event-driven perception under STARE. Compared to the frame-based approach in (b), STARE's Continuous Sampling enables higher throughput, reducing temporal misalignment between model predictions (blue bounding box) and ground truth (green bounding box). This visualization conceptually demonstrates the temporal advantage of Continuous Sampling.

an event is triggered to the moment a downstream application receives the perception model's corresponding output, which is a critical factor in real-world deployment. In such scenarios, downstream applications such as robotic control require continuous access to the most recent outputs of perception models, and even minor latency can lead to outdated predictions and compounding errors. This mismatch between evaluation and real-time application is especially pronounced in neuromorphic vision, where event cameras generate data at over 1 MHz, yet existing frameworks process these streams as low-frequency frames (typically ~25 Hz), discarding temporal precision and penalizing lightweight, high-speed algorithms by confining them to suboptimal input frequencies.

To unlock the potential of event cameras in real-time applications, recent advances in perception frameworks have attempted to reduce latency by either optimizing frame-based processing pipelines[9,25–28] or redesigning the perception framework to directly process the online

https://doi.org/10.1038/s41467-026-70240-6





c

Dataset	Videos (train/test)	Annotations	Modality	Frequency	Resolution	Event Stream Recording Device	Time-aligned Annotation
EED (63)	5	233	Gray, Event	50	240×180	DAVIS240B (76)	✔
EVIMO (71)	34/21	77K	Gray, Event	200	346×260	DAVIS346C (77)	✔
EVIMO2 (72)	81/92	494K	RGB, Event	200	640×480	Prophesee Gen3 (78, 79) & Samsung DVS Gen3 (80)	✔
FE108 (21)	76/32	1,395K	Gray, Event	240	346×260	DAVIS346 (77)	✘
VisEvent (24)	500/320	371K	RGB, Event	25	346×260	DAVIS346	✘
COESOT (73)	827/527	478K	RGB, Event	25	346×260	DAVIS346	✘
EventVOT (5)	859/282	569K	Event	/	1280×720	Prophesee EVK4–HD (81, 82)	✘
CRSOT (74)	836/194	305K	RGB, Event	/	1280×800	CeleX-V (83)	✘
FELT (75)	730/314	1,950K	RGB, Event	25	346×260	DAVIS346	✘
ESOT500-L	146/56	1,022K	RGB, Event	500	346×260	DAVIS346	✔
ESOT500-H	74/31	475K	RGB, Event	500	1280×720	DVSync (84)	✔

Fig. 2 | The ESOT500 dataset for high-dynamic event-driven perception. **a** Representative event stream samples from the low-resolution ESOT500-L subset (346 × 260, top row) and high-resolution ESOT500-H subset (1280 × 720, bottom row), showcasing diverse high-dynamic scenarios (e.g., flag waving, bicycle riding, car driving, cap shaking, football playing, monkey swinging, fan rotating, pigeon taking off, bottle spinning, breakdancing). **b** Scene category distribution for ESOT500-L (left) and ESOT500-H (right), quantifying the percentage of sequences across attributes like object similarity, background complexity, deformation, occlusion, motion speed, and indoor/outdoor setting. **c** Comparison of event-based object tracking datasets. ESOT500-L and ESOT500-H stand out with 500 Hz time-aligned annotations, high resolution (up to 1280 × 720 for ESOT500-H), and diverse scene coverage, addressing gaps in prior datasets (e.g., low annotation frequency, lack of time-aligned labels)[5,21,24,63,71–84].

streaming data[29–32]. Despite these improvements, evaluation methodologies remain predominantly based on fixed frame-rate sequential data. To address this issue, we introduce the STream-based lAtency-awaRe Evaluation (STARE) framework, which is designed to transcend traditional limitations and realistically assess event-driven models through two core components, as shown in Fig. 1c. First, Continuous Sampling schedules the model to immediately process the latest events right after the previous cycle, maximizing model throughput for real-time use. Second, Latency-Aware Evaluation integrates latency into accuracy metrics: it aligns each timestamped ground truth with the latest available prediction, directly quantifying accuracy degradation caused by outdated model outputs. An example of the perception result from STARE is shown in Fig. 1d.

Given that existing event datasets are constrained by low frame-rate annotations and designed for the traditional latency-ignored framework, we developed a new event dataset with high-frequency annotations: ESOT500[33], a 500 Hz object tracking dataset in which each annotation simulates a real-world downstream query (Fig. 2). This annotation density ensures resistance to temporal aliasing[34] and supports rigorous Latency-Aware Evaluation (Fig. 3). Leveraging STARE with ESOT500, we demonstrate that perception latency reduces accuracy by over 50% (Fig. 4a–b). Robotic experiments further validate this finding: a 55% increase in perception latency leading to complete task failure (Fig. 5).

To address the accuracy degradation stemming from latency, we propose two model enhancement strategies that underscore the need for advanced architectural designs and operational paradigms for event-driven perception systems (Fig. 6). First, Asynchronous Tracking, which draws inspiration from harnessing the continuity of event streams to improve model throughput and thus fulfills the dense query requirements from downstream applications[9,25], adopts a dual-component architecture: a heavyweight base model generates high-accuracy yet low-frequency predictions, while a lightweight residual model is integrated to rapidly refine the most recent output of the base model. Second, Context-Aware Sampling, which originates from our key observation that model performance correlates with the event density surrounding the target object. This method dynamically adjusts the model's activation state based on this contextual event

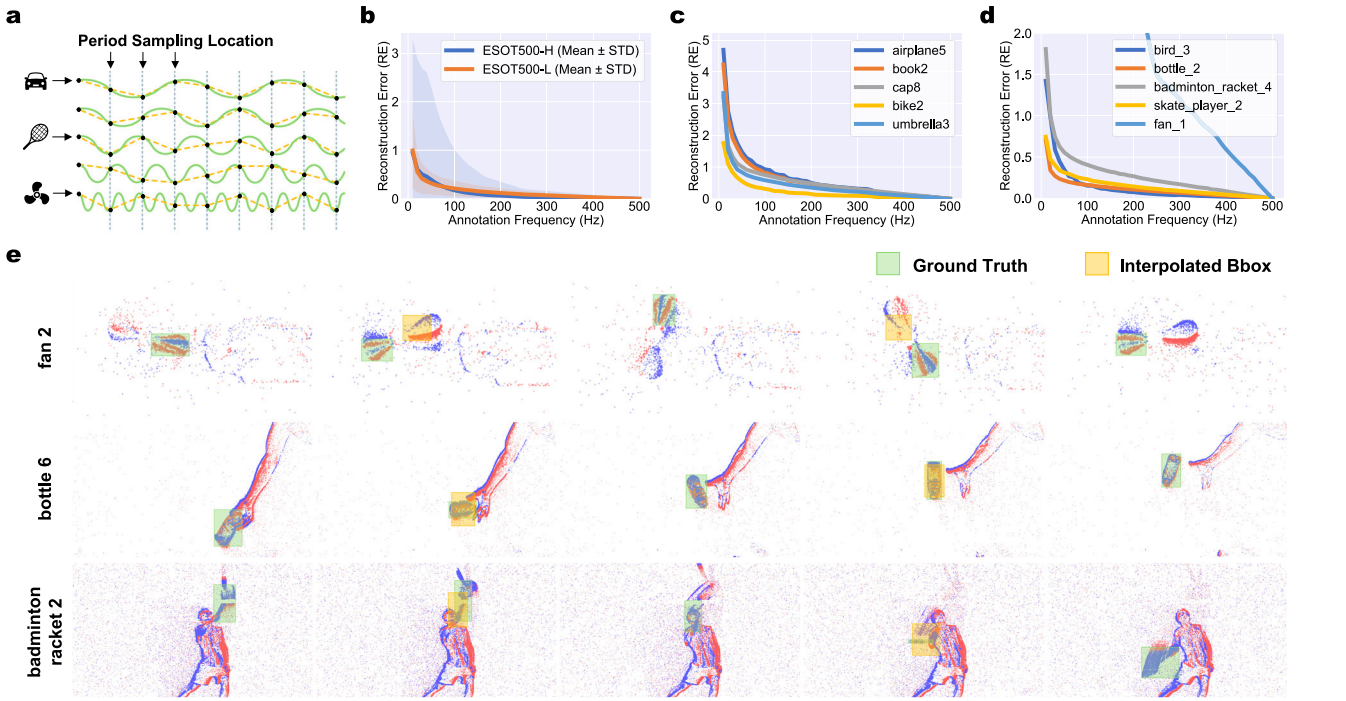


Fig. 3 | Temporal aliasing in event-driven perception and ESOT500's solution. **a** Conceptual illustration of temporal aliasing. The green solid line denotes the true continuous object state over time; black dots are low-frequency periodic annotations; the yellow dashed line is the trajectory reconstructed from sparse annotations. For simple motion (top), reconstruction approximates the truth, but for complex motion (bottom), low-frequency sampling causes distorted reconstruction (temporal aliasing). **b** Reconstruction Error (RE, mean ± STD) for ESOT500-H (blue) and ESOT500-L (orange) as a function of annotation frequency. RE is substantial at low frequencies and gradually decreases when approaching 500 Hz, validating ESOT500's ability to mitigate aliasing via high-frequency annotation. **c** RE curves for representative sequences in ESOT500-L (airplane, book, cap, bike, umbrella). **d** RE curves for representative sequences in ESOT500-H (bird, bottle, badminton racket, skate player, fan). **e** Visual comparison of high-frequency ground truth annotations (green boxes) and sparse 20 Hz interpolated boxes (yellow) for three objects (fan, bottle, badminton racket). Interpolated boxes deviate significantly from ground truth, emphasizing the need for dense annotation to address temporal aliasing.

density. Experimental results demonstrate the efficacy of these strategies: Asynchronous Tracking improves latency-aware accuracy by up to 60% (from 31.83 to 51.06 AUC) via a 78% enhancement in model throughput (from 118 Hz to 210 Hz), whereas Context-Aware Sampling further enhances robustness, yielding over 51% performance improvement (from 18.73 to 28.29 AUC) in challenging scenarios (Fig. 7).

Our approach breaks free from the static, latency-ignored constraints of traditional vision research by centering on a critical, underaddressed principle: temporal congruence between event cameras' continuous sensing, model computation, and real-world task demands. Unlike prior work that forces event streams into frame-based pipelines or treats latency as an isolated metric, we present a cohesive solution that integrates STARE (a stream-based latency-aware evaluation framework that mirrors real deployment), ESOT500 (a dense 500 Hz dataset for validating the framework), and our asynchronous/context-aware strategies, thereby narrowing the divide between the theoretical speed of event cameras and their practical utility in real-time scenarios. This represents the main contribution of our work.

Results

The stream-based latency-aware evaluation framework

We developed the STream-based lAtency-awaRe Evaluation (STARE) framework: a realistic, rigorous benchmarking tool explicitly tailored to event-driven vision systems, designed to transcend the limitations of conventional latency-ignored paradigms. STARE's design is anchored in two core, mutually reinforcing principles that govern both the operation of models in online streaming scenarios and the quantification of their performance under real-time constraints, namely Continuous Sampling and Latency-Aware Evaluation.

Continuous sampling in STARE. The traditional framework operates offline by converting a continuous event stream into a sequence of fixed-rate frames and processing them sequentially, as shown in Fig. 1b. However, adapting this fixed-rate method for online operation would disrupt the event stream's continuity, lowering model throughput and creating inefficiency. The core issue is the inherent misalignment between a model's variable inference latency and the fixed frame interval. This misalignment inevitably causes idle time for both faster and slower models, as they must wait for the next frame to arrive, thereby artificially limiting maximum throughput[10–13]. More details can be found in Section "Formal Definition".

To address this, STARE employs the Continuous Sampling strategy during the model's perception stage. As shown in Fig. 1c, this approach leverages the continuity of the event stream by scheduling the model to sample the most recent events for processing immediately upon completing the prior processing cycle. As illustrated by the comparison between Fig. 1b, d, Continuous Sampling eliminates idle time and enables the model to operate at maximum throughput, which is critical for real-time applications such as robotic control or high-speed interaction.

Latency-aware evaluation in STARE. The second core principle of STARE lies in its evaluation protocol, which explicitly integrates perception latency into the final accuracy metrics. As shown in Fig. 1b, the traditional framework evaluates a model's prediction against the ground truth at the input's timestamp, implicitly assuming that predictions are available instantaneously[6,7,35–39]. Such an assumption

https://doi.org/10.1038/s41467-026-70240-6

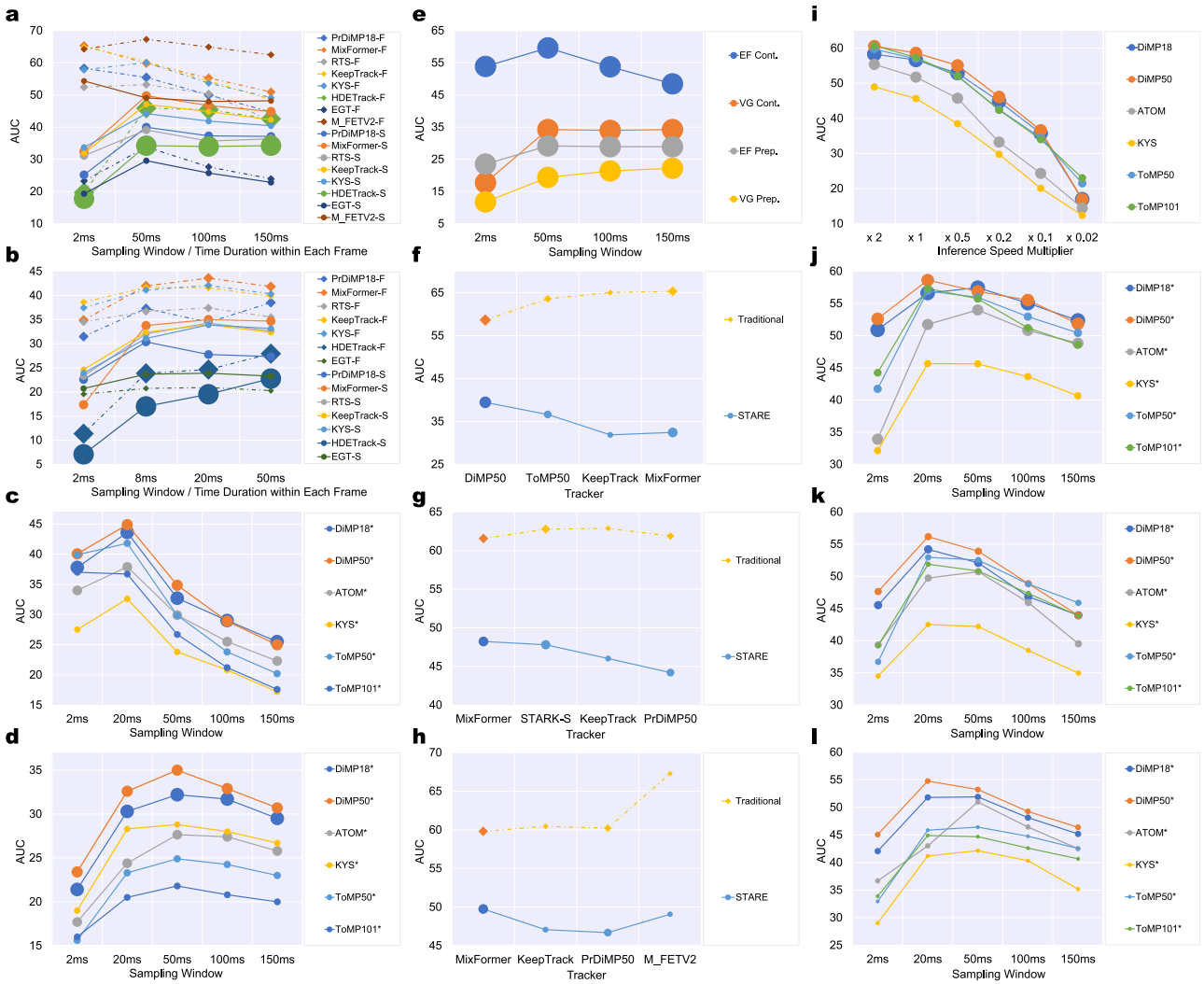


Fig. 4 | Impact of perception latency on event-driven trackers across datasets and hardware. a Tracking accuracy (AUC) under STARE (-S, solid lines) vs. traditional frame-based evaluation (-F, dashed lines) on ESOT500-L. M_FETV2 stands for Mamba FETrackV2. Larger markers indicate faster inference. STARE reveals the impact of perception latency on accuracy hidden by traditional methods. Accuracy generally peaks at an optimal sampling window size under STARE. **b** Same as (**a**) but on ESOT500-H, validating STARE's consistency across dataset resolutions. The unimodal AUC vs. sampling window size trend persists. **c** STARE performance on FE108[21]: AUC vs. sampling window size for diverse trackers. The consistent unimodal trend across datasets (vs. ESOT500) highlights STARE's generalizability to external benchmarks. **d** STARE performance on VisEvent[24]: AUC vs. sampling window size. Results mirror (a-c), reinforcing the unimodal trend. **e** Ablation of sampling methods with HDETrack: Continuous Sampling (STARE, "Cont.") vs. fixed-rate sampling from preprocessed frames ("Prep."), using EventFrame (EF)[21] and VoxelGrid (VG)[21] representations. Continuous Sampling outperforms, leveraging event stream temporal continuity. Performance ranking reversals under STARE at 2 ms (**f**), 20 ms (**g**), and 50 ms (**h**) sampling window sizes. Faster models (e.g., DiMP50, MixFormer) outperform slower counterparts with high traditional accuracy, demonstrating STARE's bias toward throughput. **i** Accuracy degradation with simulated inference latency (speed multiplier < 1 slows inference). All models show monotonic AUC drops, quantifying latency's direct impact. STARE performance on hardware with varying configurations: (**j**) RTX 3090 (high-power), (**k**) RTX 3080 Ti (lower-power, accuracy drop vs. 3090), (**l**) RTX 3080 Ti with parallel task contention (further degradation), illustrating how different hardware configurations impact on latency.

neglects the inevitable delay between sensory input and computational output, thereby overlooking the critical influence of perception latency on real-time performance[9–13].

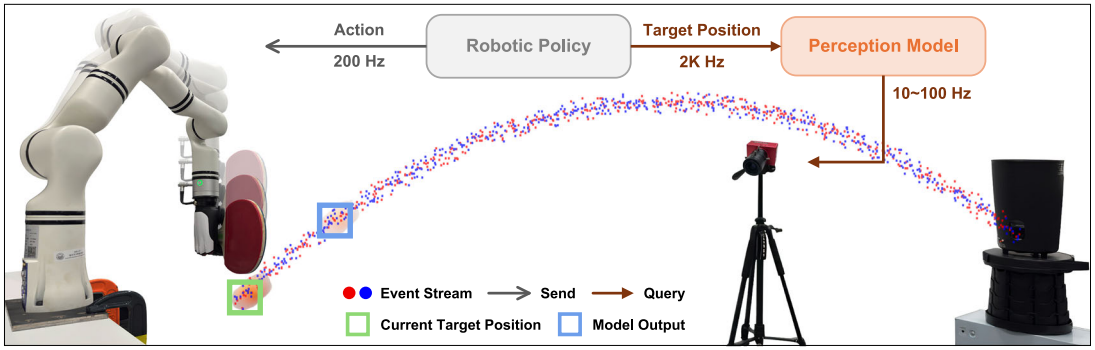
To address this, as shown in Fig. 1c, STARE employs a Latency-Aware Evaluation protocol, which simulates the interaction between the perception model and the downstream application. The core of this protocol involves comparing sparse model outputs against a dense sequence of high-frequency ground truth annotations, with each annotation representing a real-time query. For any query at time t_{query}, the most recent perception result prior to that time (i.e., $t_{output} \leq t_{query}$) is retrieved to calculate the performance metric (e.g., AUC, IoU).

This mechanism inherently highlights the impact of latency. When a model exhibits high latency, its sparse outputs result in a single prediction being repeatedly utilized as the best available estimate for a sequence of queries. For example, as shown in Fig. 1c, the perception result generated at time t_{i+2} is reused to evaluate queries after t_{i+2}, such as t_j where $t_{i+2} < t_j < t_{i+3}$, thereby accumulating error over time. This approach provides a direct quantification of the performance degradation attributable to perception latency, yielding a faithful assessment of a model's suitability for real-time deployment.

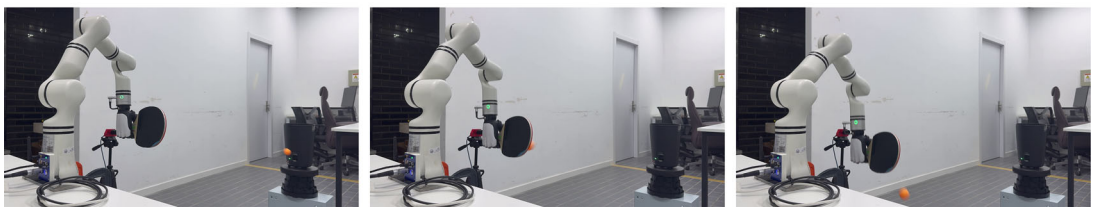
The stream-based latency-aware evaluation dataset

To enable rigorous validation of the STARE framework and address the limitations of existing event datasets, which lack dense temporal annotations for Latency-Aware Evaluation, we introduce the ESOT500 dataset. ESOT500 comprises two subsets with distinct sensor resolutions: ESOT500-L (346 × 260) and ESOT500-H (1280 × 720), covering

a



b



c

Model	ESOT500-L Performance (AUC)		Speed (Hz)			Success Rate
	Latency-ignored	Latency-aware	Mean	Min/Max	STD	
MixFormer	61.56	48.23	55.3	10.6/95.0	17.2	7/20
			36.9	17.5/59.5	8.8	5/20
			23.0	12.4/37.0	4.7	0/20
MixFormer*			39.9	19.0/83.7	9.2	2/20
KeepTrack	62.87	46.02	33.2	20.9/44.1	4.3	1/20

Fig. 5 | Real-world robotic ping-pong experiment. a Perception-action loop experimental setup. An event camera captures the ping-pong ball trajectory (event stream, red/blue points) at 1 MHz. An upstream tracker (orange) runs at tens of Hz, outputting bounding boxes (blue) to a downstream robotic policy (gray). The policy queries for target positions at 2 kHz and sends control actions to the robotic arm at 200 Hz. **b** An example of a successful robot hitting back a ping-pong ball. **c** Robotic ping-pong task success rate across perception models. Metrics include: ESOT500-L performance (AUC, latency-ignored vs. latency-aware), model speed (Hz, mean, min/max, STD), and success rate (successful hits / 20 trials). Underlined values indicate column maxima. MixFormer (high speed) achieves the highest success rate (7/20), while frame-based variants (e.g., MixFormer*) or high-offline-accuracy/low-speed models (KeepTrack) show reduced success, validating latency's critical role in real-time task.

diverse indoor/outdoor scenes, object classes, and challenging environmental conditions (Fig. 2a–b). A comparative analysis with related event datasets is presented in Fig. 2c, while detailed protocols for data capture, annotation, training/test splits, and implementation are provided in Supplementary Note 3. Latency-Aware Evaluation (a core goal of STARE) requires two critical dataset attributes, as identified in prior analysis: (1) dense ground truth to simulate the high-frequency queries of real-world downstream applications (e.g., robotic control)[40–44], and (2) high temporal resolution to capture high-dynamic object motion without the temporal aliasing inherent in low-frequency annotations[34,45]. ESOT500 is purpose-built to satisfy both requirements via its 500 Hz annotation rate, as elaborated below.

Simulating dense downstream queries with 500 Hz annotations. The 500 Hz annotation frequency, which is ESOT500's defining feature, generates a dense ground truth sequence that mimics the continuous, high-rate query demands of downstream applications[40–44]. This enables STARE to directly assess latency-induced accuracy: by comparing a model's sparse, timestamped outputs with ESOT500's dense query stream, the framework captures how latency causes predictions to become outdated relative to real-time task demands, which would be impractical with low-frequency datasets.

High-fidelity capture of high-dynamic object motion. The 500 Hz annotation rate enables high-fidelity capture of high-dynamic object motion, addressing a critical shortcoming of low-frequency datasets, which struggle to resolve rapid, transient motion transitions. From an information-theoretic perspective, sampling continuous high-dynamic motion at discrete, low frequencies acts as a lossy channel[45]: the original fast-changing trajectory serves as the input signal, while sparse annotations produce degraded outputs that omit key motion details. As the speed or variability of object motion increases, this degradation becomes more severe (Fig. 3a)[34], leading to substantial discrepancies between the true high-dynamic motion and trajectories interpolated from low-frequency sparse annotations (Fig. 3e).

To quantify this distortion, we introduce the Reconstruction Error (RE) metric, which measures motion information loss at low sampling frequencies by: (1) downsampling ESOT500's 500 Hz ground truth to a target frequency; (2) reconstructing the trajectory via linear interpolation; and (3) computing the error relative to the original 500 Hz data. As shown in Fig. 3b–d, RE is pronounced at conventional low frequencies (e.g., 25 Hz) but gradually declines as the sampling rate approaches 500 Hz. Fig. 3e presents several visual examples. These results confirm that high temporal resolution is critical to preserve the integrity of high-dynamic motion. The RE curve is expected to be even steeper in more complex tasks (e.g., multi-object detection in dynamic scenes[6], 6D pose estimation for high-speed targets[25], visual odometry in rapid locomotion[7]), as higher degrees of freedom introduce greater uncertainty. These results provide quantitative evidence that ESOT500's high-frequency annotations are crucial for validating

https://doi.org/10.1038/s41467-026-70240-6

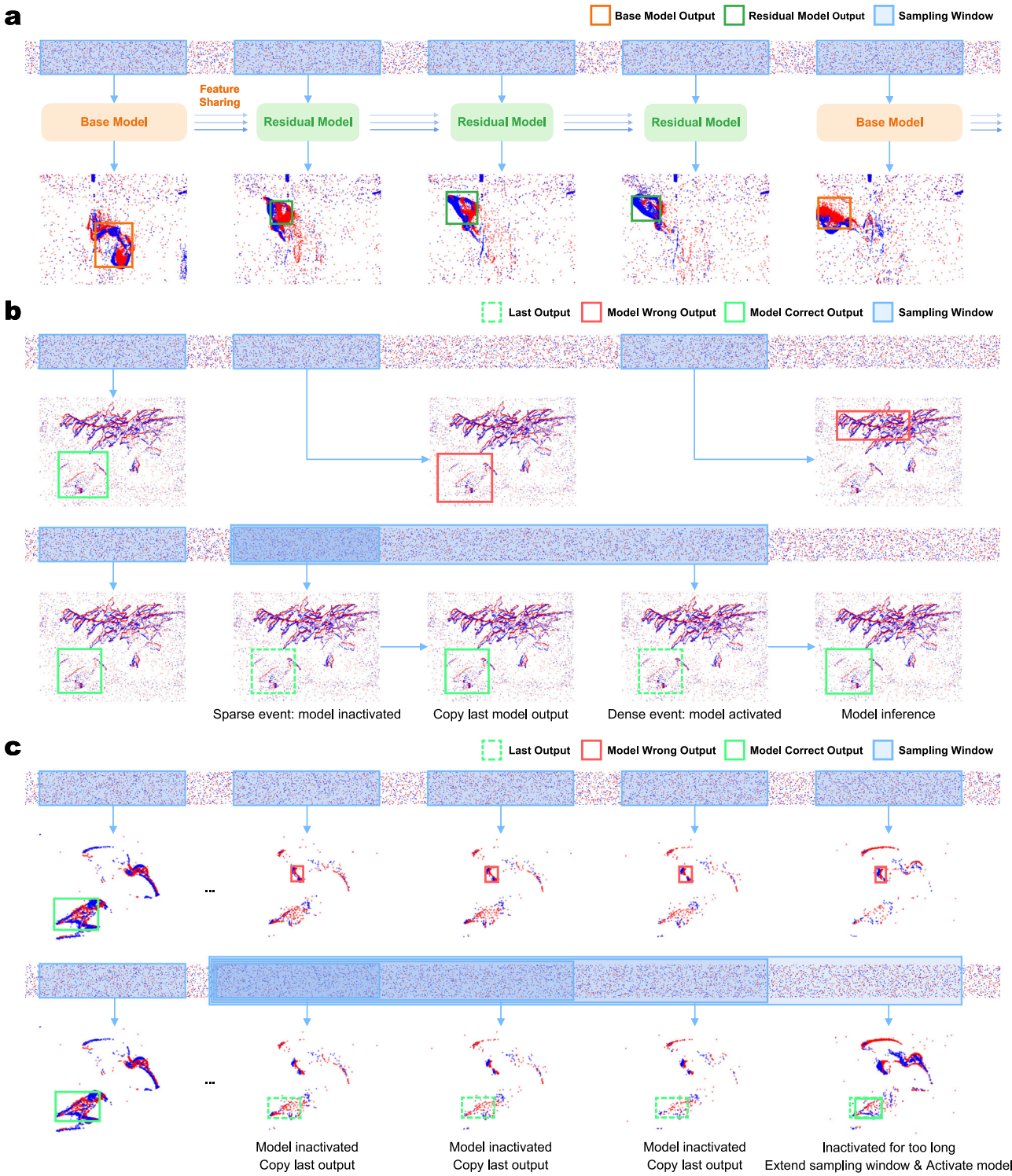


Fig. 6 | Conceptual illustration of the proposed Asynchronous Tracking and Context-Aware Sampling. a Architecture of Asynchronous Tracking. A slow, high-fidelity base model (orange) performs full inference on event segments, generating initial bounding boxes and sharing features with a fast residual model (green). The residual model recursively updates predictions using shared features and new events, producing high-frequency outputs between base model cycles, leveraging temporal continuity of event stream to boost throughput. **b** Qualitative example of Context-Aware Sampling in sparse-event scenarios. Top row: Baseline model fails to localize the target (red box) as event density drops. Bottom row: Enhanced model detects sparse events, enters an inactive state, and reuses the last correct prediction (dashed green box) until dense events trigger accurate inference, preventing error accumulation. **c** Qualitative example of Context-Aware Sampling mitigating target drift during prolonged inactivity. Top row: Baseline tracker accumulates errors over time and loses the target. Bottom row: Enhanced tracker uses a timer to force reactivation after prolonged inactivity, re-localizing the slowly drifting target.

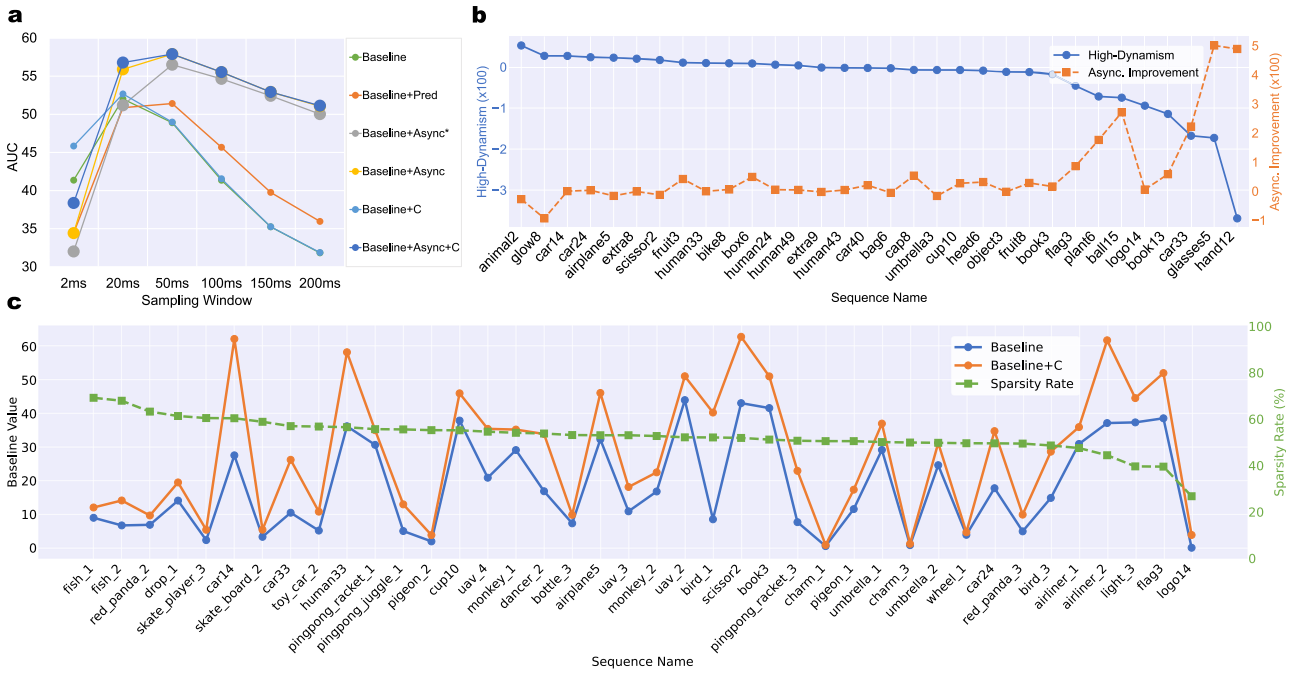


Fig. 7 | Quantitative evaluation of model enhancement strategies under STARE. **a** Accuracy (AUC) of OSTrack-based[85] variants on ESOT500-L across sampling window sizes. Curves compare: Baseline (green), +Predictive Motion Extrapolation (+Pred, orange), +Context-Aware Sampling (+C, light blue), +Asynchronous Tracking (trained on 500 Hz annotations, yellow), and +Asynchronous Tracking (trained on 20 Hz annotations, gray). Asynchronous Tracking (500 Hz) combined with Context-Aware Sampling (+Async+C, dark blue) consistently outperforms other strategies. **b** Motion dynamism vs. Asynchronous Tracking effectiveness. Blue solid line: high-dynamism (left y-axis, negative performance gain from Predictive Motion Extrapolation, also defined as the unpredictability score) for ESOT500-L sequences. Orange dashed line: Accuracy improvement (right y-axis) from Asynchronous Tracking. Higher dynamism poses greater challenge. **c** Context-Aware Sampling performance in some sparse-event scenarios. Blue line: Baseline accuracy (left y-axis). Orange line: Accuracy with Context-Aware Sampling (left y-axis). Green dashed line: Sparsity Rate (right y-axis, percentage of model inactivity). Context-Aware Sampling demonstrates robustness in low-motion contexts.

event-driven models in high-dynamic scenarios. A formal definition of RE is provided in Section "Reconstruction Error" (Methods).

Experimental results and analysis

Event-driven perception evaluation with STARE. To quantify the impact of perception latency on event-driven perception, we leverage the STARE framework to evaluate state-of-the-art models. This evaluation centers on the ESOT500 dataset, tailored for high-dynamic, low-latency assessment, and extends to external benchmarks to verify generalizability.

Experimental setting. We selected representative trackers spanning diverse model families, offline accuracy, and inference speeds (Fig. 4a–b): Siamese-based (PrDiMP18[46]), Transformer-based (MixFormer[47]), GNN-based (KeepTrack[48]), RNN-based (KYS[49]) and Segmentation-Centric (RTS[50]). We also incorporated models specifically designed for event streams, such as those operating directly on raw events (EGT[29]), grid-like representations[21] (HDETrack[5]), and a recent cross-modal tracker using the Mamba architecture (Mamba-FETrackV2[51]). Trackers were assessed under two paradigms: (a) Traditional framework: Event streams were preprocessed into 20 Hz fixed-rate frames (with varying event durations per frame)[22]. (b) STARE framework: Trackers used Continuous Sampling, processing the most recent events within different sampling window sizes. More details about the experimental settings can be found in Supplementary Note 1.

Evaluation on ESOT500. The results, summarized in Fig. 4a–b, reveal two key findings. First, most models experienced a substantial accuracy drop by up to 50% when moving from the traditional framework to STARE. This demonstrates that accuracy degradation caused by perception latency has been largely overlooked by the traditional evaluation framework. While the accuracy drop was generally observed, the results under STARE highlighted the superiority of some lightweight models over heavy ones. As shown in Fig. 4f–h, some models that achieved higher offline accuracy in the traditional framework were outperformed by more lightweight models under STARE. For example, MixFormer[47] outperformed KeepTrack[48] under STARE, whereas KeepTrack performed better in the traditional framework. This result is consistent with our real-world robotic ping-pong experiment in Fig. 5c. Notably, such performance ranking reversals are observed even on our RTX 3090 GPU with high computational power. It is reasonable to infer that the impact of latency is much more pronounced on resource-constrained devices, which are common in robotics and autonomous systems[52,53]. These findings provide direct experimental evidence that the traditional framework can yield misleading conclusions about a model's practical capability, underscoring the necessity of a latency-aware evaluation paradigm such as STARE.

Second, the performance of most trackers under STARE followed a unimodal trend with respect to the sampling window size. The average peak performance appeared around a 20 ms window, reflecting a trade-off: longer windows gather more information but risk redundancy, while shorter windows avoid redundancy but miss critical data. Drawing on this finding, we noticed the effectiveness of Continuous Sampling was influenced by sampling window size. Hence, we made a deeper observation on Continuous Sampling by comparing the model performance under STARE and a frame-based latency-aware evaluation. Under the frame-based latency-aware evaluation, the event stream was converted into event frames in advance, and the model was constrained to sample events only at the discrete timestamps of these frames. More details about the frame-based latency-aware evaluation

can be found in Section "Formal Definition" (Method). In contrast, under STARE with Continuous Sampling, the model sampled and processed the most recent events immediately after the previous processing was complete, not requiring any fixed-rate preprocessing. We evaluated model under these two settings with varying sampling window size. As shown in Fig. 4e, Continuous Sampling improved the model's online accuracy by 51–129% with various sampling windows, demonstrating that Continuous Sampling is able to improve the model throughput thereby enhances accuracy by leveraging the continuity of event data. Furthermore, Continuous Sampling worked the most effectively with a sampling window of 50 ms, which best balanced window size and information richness on our dataset. It revealed that the window size in Continuous Sampling should be carefully considered to unlock a model's full real-time potential.

The above experiments highlighted the impact of latency on model performance. To comprehensively evaluate a model under STARE in different latency conditions, we conducted experiments using a latency simulator, different physical GPU&CPU configurations, and resource contention from a parallel task. The results are shown in Fig. 4i–l, respectively. In all cases, we found that the accuracy dropped consistently under STARE as latency increased, demonstrating the consistent negative impact of perception latency on model performance.

Evaluation on other datasets. Apart from the proposed ESOT500 dataset, we also applied STARE to two other commonly-used event-based tracking datasets: FE108[21] and VisEvent[24]. As shown in Fig. 4c–d, the performance of models on these datasets also followed the unimodal distribution with respect to the sampling window size, similarly to our findings on ESOT500. It is worth noting that the optimal sampling window for Continuous Sampling on FE108 (e.g., 20 ms for KYS[49]) is much shorter than that on VisEvent (e.g., 50 ms for KYS). This is likely because the objects in FE108 are hung on a stick and swung, generating high-density events. As a result, a short sampling window in this case would lead to more proper information aggregation.

Evaluation on ping-pong robot player with STARE. To examine the real-world impact of perception latency, we developed an event-based ping-pong robot player. This platform enabled us to test the immediate impact of latency on applications that demand rapid responses. The system consisted of a robotic arm holding a bat, an event camera, a ball launcher, an event-driven tracker, and a robot control policy. These components formed a tightly coupled perception-action loop. The event camera recorded the ball's trajectory at 1 MHz. The tracker, operating at tens of Hz, requested the latest event data and provided estimated 3D positions of the ball. The robot control policy queried the tracker at 2 kHz, computing hitting trajectories, and sent control commands to the arm at 200 Hz, guiding it to strike the ball. The complete setup is illustrated in Fig. 5a. An example of a successful robot hitting back a ping-pong ball is shown in Fig. 5b. More details are provided in Supplementary Note 2.

To quantify the impact of perception latency on the ping-pong robot player, we conducted experiments under five conditions with results summarized in Fig. 5c. In the first three settings, we applied STARE to the MixFormer[47] model and simulated different levels of perception latency by varying the model processing speed. Increasing the processing speed from 23.0 Hz to 55.3 Hz improved the success rate from 0 to 7 out of 20 trials, demonstrating that reduced latency contributed to real-world task success. The fourth setting served as a critical ablation study to STARE by removing Continuous Sampling operation. When the event stream was converted into 40 Hz frames, the model success rate dropped to 2 out of 20 trials, lower than the second setting (5 out of 20) with a close processing speed but with Continuous Sampling operation. This demonstrated that the traditional frame-based online setting failed to unlock the model's real-time potential. In the final setting, we selected KeepTrack[48] that has a higher traditional offline accuracy than MixFormer (62.87 AUC vs 61.56 AUC) but has a lower processing speed (33.2 Hz). KeepTrack achieved a lower task success rate (1 out of 20 trials), highlighting the negative impact of high processing latency even though the model had a higher offline accuracy.

Overall, these findings emphasize two essential requirements for evaluating event-driven systems. First, perception latency should be explicitly considered in evaluation to ensure that performance metrics reflect realistic deployment conditions rather than idealized assumptions. Second, to fully realize the capabilities of event-driven models, the evaluation methodology should preserve the continuous nature of event streams instead of constraining them into fixed-rate frames, which can suppress real-time potential even when the nominal throughput appears high.

Strategies for continuous event-driven perception

To mitigate latency-induced performance loss while aligning with the intrinsic continuity of event streams, we propose two targeted model enhancement strategies: Asynchronous Tracking (boosts throughput via dual-model collaboration) and Context-Aware Sampling (optimizes input efficiency via dynamic event selection).

Asynchronous tracking. Increasing model throughput is a direct way to produce more timely predictions, which is critical for satisfying the high-frequency query demands from downstream applications (e.g., robotic control). Event streams inherently provide continuous visual information at near-arbitrary timestamps, theoretically supporting ultra-high throughput. But this potential is not fully utilized by single-model architectures, which are limited by the computational latency of feature extraction. To leverage this continuity, we develop Asynchronous Tracking: a dual-model framework that uses a lightweight residual model to recursively update predictions from a heavyweight base model, thereby boosting throughput without sacrificing accuracy.

Figure 6a details the architecture: the heavyweight base model extracts high-quality features from the event stream to estimate target positions, but results in significant computational latency. While the base model runs inference, the lightweight residual model processes the latest incoming events to refine the base model's features and predictions in real time. This design has two key advantages: it avoids redundant, computationally expensive feature extraction (by reusing the base model's shared features) and integrates the most recent event information (preserving temporal continuity).

Experimental validation (Fig. 7a) confirms the strategy's efficacy: Asynchronous Tracking increases model throughput by 78% (from 118 to 210 Hz) and lifts latency-aware accuracy by 60% (from 31.83 to 51.06 AUC). For comparison, we also explored a single-model alternative, the Predictive Motion Extrapolation, which extrapolates future target positions via velocity vectors output by the base model (details in Method Section "Predictive Motion Extrapolation"). Both strategies aim to boost throughput by updating base model predictions, but Asynchronous Tracking outperforms Predictive Motion Extrapolation across all settings (Fig. 7b). We attribute this performance gap to the limitations of motion extrapolation in high-dynamic scenarios, particularly critical for ESOT500's 500 Hz annotated high-dynamic motion. To quantify this, we define the unpredictability score (negative accuracy gain of Predictive Motion Extrapolation relative to a non-predictive baseline), where higher scores indicate more unpredictable, namely high-dynamic motion. As shown in Fig. 7b, a model achieves sharply decreased accuracy while unpredictability scores increase, confirming why predictive motion extrapolation underperforms.

Context-aware sampling. Our earlier STARE evaluations (Section "Event-Driven Perception Evaluation with STARE") revealed a key insight: model performance exhibits a unimodal distribution with respect to event sampling window size. Specifically, too small a window lacks sufficient motion information, while too large a window introduces redundant information. This finding motivated Context-Aware Sampling: a dynamic input refinement strategy that adapts the sampling window while activating/deactivating model inference based on event density surrounding the target's last known location. The goal is to balance computational efficiency (avoiding unnecessary and inaccurate inference) and perception accuracy (preventing the target from slowly drifting).

Qualitative examples in Fig. 6b–c illustrates the mechanism: When event density is low (indicating slow or no motion, shown in Fig. 6b), inference is deactivated, and the last valid prediction is reused. This can reduce computational latency without accuracy loss. To avoid target drift from prolonged deactivation (Fig. 6c), the strategy enforces periodic reactivation to re-localize the target using newly accumulated events. The detailed formulation of the event density threshold and reactivation interval is provided in Section "Context-Aware Sampling" (Method). Experimental results (Fig. 7a, c) confirm its effectiveness: Context-Aware Sampling improves performance across most settings, and when combined with Asynchronous Tracking, achieves a 61% accuracy gain (from 31.83 to 51.12 AUC), as shown in Fig. 7a. Its effectiveness is most pronounced in sparse-event scenarios. Fig. 7c shows the strategy lifts accuracy by over 51% from an average of 18.73 to 28.29 AUC, demonstrating its robustness in challenging scenarios with sparse events.

Discussion

Neuromorphic vision systems hold unique potential for real-time applications, yet their practical deployment has been hindered by misalignment between evaluation paradigms and the intrinsic properties of event streams. To address this critical gap, we developed the STream-based lAtency-awaRe Evaluation (STARE) framework, which is designed to prioritize the temporal continuity of event data and explicitly account for perception latency, two factors overlooked by traditional RGB-derived evaluation methods.

STARE's value lies in its ability to bridge theoretical benchmarks and real-world performance. By integrating Continuous Sampling and Latency-Aware Evaluation, the framework aligns model operation pipeline with the asynchronous nature of event streams: Continuous Sampling maximizes throughput by processing the latest events immediately after the prior processing cycle, while Latency-Aware Evaluation quantifies latency-induced accuracy loss by matching high-frequency ground truth to the most recent available model output. This design ensures STARE captures performance degradation that traditional frame-based, latency-ignored framework misses. Our experiments show perception latency reduces online accuracy by over 50%, and even reverses model rankings (favoring lightweight, high-throughput architectures over computationally heavy alternatives). Critically, event-driven robotic tests validated this finding: a 55% increase in latency can lead to a complete (100%) drop in task success rate (ping-pong return accuracy), underscoring the consequences of ignoring latency in real-world systems.

To enable rigorous validation of STARE, we developed the ESOT500 dataset, which provides 500 Hz dense annotations to accurately capture high-dynamic object motion and avoid temporal aliasing. This dataset addresses a key limitation of existing low-frequency event datasets, which fail to faithfully capture high-dynamic state changes and thus struggle with providing reliable Latency-Aware Evaluation results. Complementing STARE and ESOT500, we proposed two model enhancement strategies: Asynchronous Tracking and Context-Aware Sampling that mitigate latency-induced degradation without sacrificing model computational efficiency. Asynchronous Tracking boosts throughput via a dual lightweight-heavyweight architecture, while Context-Aware Sampling dynamically adapts input based on event density surrounding the target object. The two strategies together improve latency-induced accuracy by up to 61% and increase model speed by 78%. Furthermore, Asynchronous Tracking outperforms alternative approaches like Predictive Motion Extrapolation, which struggles in highly dynamic scenarios due to its reliance on trajectory forecasting rather than improving throughput by leveraging the continuous event stream.

Despite these advances, our work has limitations that point to future research directions. First, while our vision is for the STARE framework to be applied across a wide range of perception tasks, our current work demonstrates its methodology on the single-object tracking task, chosen for its clarity as a foundational perception problem. In turn, this work paves the way for STARE's application to other demanding real-time tasks, as more high temporal resolution datasets tailored to these scenarios are established in the future. Moreover, we currently treat upstream perception modules and downstream applications (e.g., robotic control) as decoupled systems, focusing on how perception latency impacts control outcomes rather than simultaneously optimizing them as a whole. A more integrated approach would involve co-designing perception and control policies within a unified latency-aware framework. Recent work has explored learning a separate high-level controller to make runtime decisions[54] or directly forecasting future states to provide better state estimation[55], effectively creating reactive policies that adapt to dynamic environments. But end-to-end policies probably offer even greater potential. For example, such policies could learn to prioritize safer, lower-risk actions when high perception latency is anticipated, which can make systems more efficient, and more robust to latency-induced uncertainty.

Looking forward, STARE also opens avenues to explore broader questions in neuromorphic vision, such as co-design of algorithms and hardware, real-time robotic control, and learning-based event sampling strategies. By centering temporal congruence in model operation and evaluation, our work provides a path towards unlocking the full potential of event-driven systems, enabling technologies that are accurate, low-latency, and reliable in real-world scenarios.

Methods

In this section, we employ VOT as a concrete example to formally describe the STARE framework and compare it with latency-ignored and frame-based approaches.

Framework

Formal definition

Preliminary and Notation. Given an event stream

$$\mathcal{E} = \{e_i\}_{i=0}^{N_e}, \tag{1}$$

we regard the sampled event stream segment as

$$\mathcal{E}_{l,r} = \{e_i\}_{i=l}^{r}, \tag{2}$$

where $e_i = (\mathbf{x}_i, t_i, p_i)$, including event coordinate, timestamp and polarity; N_e is the max event index and $N_e + 1$ is the total events number of an event stream; l, r are event indexes, corresponding to the left and right bounds of the sampled event stream segment. In this work, the basic sampling strategy can be divided into two types, one fixes the time duration of sampled event stream, while another fixes the sampled event number. Following that, the indexes l, r can be achieved by two indexing functions:

1) Given the window size of sampling time duration L, the time-fixed indexing function φ_τ can be defined as

$$[l_\tau, r_\tau] = \varphi_\tau(\mathcal{E}, t, L), \tag{3}$$

where

$$r_\tau = \max\{i | (\mathbf{x}_i, t_i, p_i) \in \mathcal{E}, t_i \le t\}, \tag{4}$$

$$l_\tau = \min\{i | (\mathbf{x}_i, t_i, p_i) \in \mathcal{E}, t_i \ge \max(t - L, t_0)\}. \tag{5}$$

2) Given the window size of sampled events number N, the number-fixed indexing function φ_μ can be defined as

$$[l_\mu, r_\mu] = \varphi_\mu(\mathcal{E}, t, N), \tag{6}$$

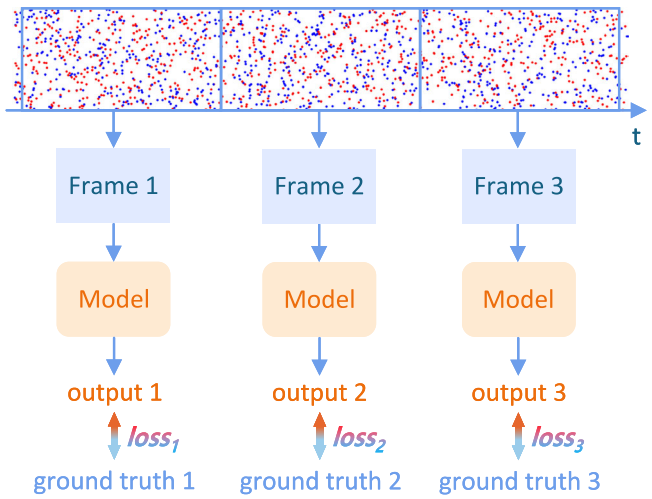


Fig. 8 | The traditional frame-based latency-ignored evaluation framework. This diagram illustrates the traditional pipeline for evaluating event-driven perception models. A continuous event stream is first preprocessed into fixed-rate event frames. Each frame is then fed into the model. The model's output is used to calculate the deviation against the ground truth at the input timestamp, ignoring the model's perception latency.

where

$$r_\mu = \max\{i | (\mathbf{x}_i, t_i, p_i) \in \mathcal{E}, t_i \le t\}, \tag{7}$$

$$l_\mu = \max(r_\mu - N, 0). \tag{8}$$

Before feeding the sampled event stream $\mathcal{E}_{l,r}$ into the perception model, we need to convert it to an event frame with a specific event representation that the model can process. The general conversion process is denoted by the function χ

$$\widehat{\mathcal{E}}_{l,r} = \chi(\mathcal{E}_{l,r}). \tag{9}$$

As to the relevant event representations, we make a brief description in Supplementary Note 1.

As illustrated in Section "Results", to meet the Latency-Aware Evaluation paradigm, each ground truth bounding box $\mathbf{B}_j$ within an event stream sequence is attached with a timestamp t_j. Specifically, we assume that the time starts from zero, and the total time duration of an event stream is T. We mark the ground truth set as

$$\mathcal{Q} = \{(\mathbf{B}_j, t_j)\}_{j=0}^{N_{gt}}, \tag{10}$$

where N_{gt} is the max ground truth index and $N_{gt}+1$ is the total number of ground truth and $\mathbf{B}_0$ serves as the template bounding box for tracking. If the sampling frequency is H segments per second, we can calculate that

$$N_{gt} = \lfloor T \cdot H \rfloor - 1. \tag{11}$$

Frame-based Latency-ignored Evaluation. As discussed in Section "Introduction" and illustrated in Fig. 8, frame-based latency-ignored evaluation involves processing preprocessed event frames one by one to calculate the offline accuracy. We mark the sequence of event frames as

$$\widehat{\mathcal{E}}_f = \{\widehat{\mathcal{E}}_{l_i,r_i}\}_{i=0}^{N_f}, \tag{12}$$

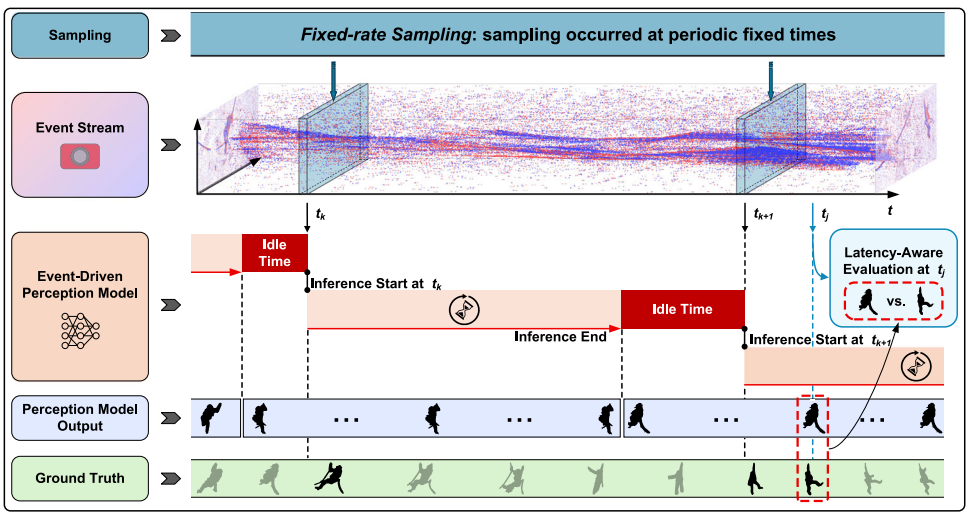


Fig. 9 | The frame-based latency-aware evaluation framework. The frame-based latency-aware evaluation framework runs models on fixed-rate frames, with event sampling restricted to periodic, fixed timestamps (e.g., t_k, t_{k+1}). This design introduces a misalignment between the model's processing latency and the frame interval, leading to unavoidable idle time periods where the model is forced to wait for the next frame before processing new events.

where N_f is the max event frame index and N_f+1 is the total number of event frames. With the introduced notations above, we can get

$$N_f = N_{\text{gt}}, \tag{13}$$

and

$$[l_i, r_i] = \varphi_\tau(\mathcal{E}, \frac{i+1}{H}, L). \tag{14}$$

Since each event frame in $\widehat{\mathcal{E}}_f$ should be fed into the model, we can get the output bounding box set described as

$$\mathcal{Q}_{\text{offline}} = \{\widehat{\mathbf{B}}_k\}_{k=1}^{N_f}, \tag{15}$$

and the corresponding output-ground-truth pairs set described as

$$\mathcal{D}_{\text{offline}} = \{(\mathbf{B}_j, \widehat{\mathbf{B}}_j) | \mathbf{B}_j \in \mathcal{Q}, \widehat{\mathbf{B}}_j \in \mathcal{Q}_{\text{offline}}\}_{j=1}^{N_{\text{gt}}}. \tag{16}$$

With $\mathcal{D}_{\text{offline}}$, we can calculate standard accuracy metrics, such as AUC and AP.

Frame-based Latency-Aware Evaluation. As illustrated in Fig. 9, frame-based latency-aware evaluation evaluates a system's reactivity to the external world by processing successively arriving frames and comparing the ground truth with corresponding timestamped outputs. The original and detailed definition can be found in ref. 10. We migrate it to the event-based vision with fixed-rate event sampling mentioned in Fig. 9. Here we provide a brief description using the previously mentioned notations.

Unlike frame-based latency-ignored evaluation, frame-based latency-aware evaluation framework does not schedule the model to process each frame in $\widehat{\mathcal{E}}_f$ one by one. If a model is currently processing the input, the coming frames will be skipped. And then, after finishing the inference, it briefly idles while waiting for the next input, as shown in Fig. 9.

The outputs contain not only the bounding boxes but also corresponding output timestamps for latency-aware evaluation. The output set can be marked as

$$\widehat{\mathcal{Q}}_{\text{streaming}} = \{(\widehat{\mathbf{B}}_k, \widehat{t}_k)\}_{k=1}^{N_{\text{streaming}}}, \tag{17}$$

where $N_{\text{streaming}}$ is the max output index and the total number of timestamped outputs. To implement the latency-aware evaluation, we still need to match each item in $\mathcal{Q}$ with an item in $\widehat{\mathcal{Q}}_{\text{streaming}}$. As shown in Fig. 9, the matching principle and the matched output-ground-truth pair set $\mathcal{D}_{\text{streaming}}$ can be described as

$$\mathcal{D}_{\text{streaming}} = \Big\{(\mathbf{B}_j, \widehat{\mathbf{B}}_{k_j}) | (\mathbf{B}_j, t_j) \in \mathcal{Q}, (\widehat{\mathbf{B}}_{k_j}, \widehat{t}_{k_j}) \in \widehat{\mathcal{Q}}_{\text{streaming}}\Big\}_{j=1}^{N_{\text{gt}}}, \tag{18}$$

where each item in $\mathcal{D}_{\text{streaming}}$ satisfies

$$k_j = \max\{k | \widehat{t}_k \le t_j, \forall (\widehat{\mathbf{B}}_k, \widehat{t}_k) \in \widehat{\mathcal{Q}}_{\text{streaming}}\}. \tag{19}$$

With $\mathcal{D}_{\text{streaming}}$, we can calculate metrics in the same way as frame-based latency-ignored evaluation, however, to show the latency-aware results.

Stream-based Latency-Aware Evaluation. As shown in Fig. 1c, in STARE, perception models get the input from continuous event streams instead of discrete frames. Algo. 1 of Supplementary Note 5 illustrates the stream-based tracking process implemented in our work.

In general, we start by setting the first ground truth bounding box $\mathbf{B}_0$'s timestamp as the current world time t_{curr}. The stream-based tracking process then begins, where in each cycle, we sample the event stream using the indexing function φ based on t_{curr} and the sampling window ω. The sampled segment $\mathcal{E}_{l,r}$ is converted into an event frame $\widehat{\mathcal{E}}_{l,r}$ in a specific representation using the conversion function χ. We then input $\widehat{\mathcal{E}}_{l,r}$ into the model along with the latest output bounding box $\widehat{\mathbf{B}}$ to predict a new bounding box and return the computational latency Δt. Then, we update t_{curr}, store the new timestamped output in $\widehat{\mathcal{Q}}_{\text{STARE}}$, and repeat the process until t_{curr} surpasses end time T of the event stream. Notably, the first cycle specifically serves to initialize the perception model with $\mathbf{B}_0$.

After the end of the tracking, we can get the timestamped bounding box list, described as

$$\widehat{\mathcal{Q}}_{\text{STARE}} = \{(\widehat{\mathbf{B}}_k, \widehat{t}_k)\}_{k=1}^{N_{\text{STARE}}}, \tag{20}$$

where the N_{STARE} is the max output index and also the total number of outputs. The method of aligning $\mathcal{Q}$ and $\widehat{\mathcal{Q}}_{\text{STARE}}$ is consistent with the frame-based latency-aware evaluation. The matched output-ground-truth pair set $\mathcal{D}_{\text{STARE}}$ can be described as

$$\mathcal{D}_{\text{STARE}} = \{(\mathbf{B}_j, \widehat{\mathbf{B}}_{k_j}) | (\mathbf{B}_j, t_j) \in \mathcal{Q}, (\widehat{\mathbf{B}}_{k_j}, \widehat{t}_{k_j}) \in \widehat{\mathcal{Q}}_{\text{STARE}}\}_{j=1}^{N_{\text{gt}}}, \tag{21}$$

where for each item in $\mathcal{D}_{\text{STARE}}$ satisfies

$$k_j = \max\{k | \widehat{t}_k \le t_j, \forall (\widehat{\mathbf{B}}_k, \widehat{t}_k) \in \widehat{\mathcal{Q}}_{\text{STARE}}\}. \tag{22}$$

We can use the same way to calculate accuracy metrics as the frame-based latency-free evaluation with $\mathcal{D}_{\text{STARE}}$, to get the perception model's performance on stream-based tracking.

Analysis. From the above description, we can see that for a given event stream $\mathcal{E}$ and sampling setting (fps and window size), both frame-based latency-ignored evaluation and frame-based latency-aware evaluation share the same event frame set $\widehat{\mathcal{E}}_f$ as input. But the frame-based latency-aware evaluation will skip some coming frames. Therefore, we can know that

$$N_{\text{streaming}} \le N_{\text{gt}} - 1, \tag{23}$$

with equality if and only if

$$\Delta t \le \frac{1}{H}, \tag{24}$$

which means the model's reasoning speed exceeds the time interval between frames. Consequently, the model will always wait for the arrival of the next frame and never skip it. When this equality holds, Eq. (19) can be simplified as

$$k_j = j - 1. \tag{25}$$

On the contrary, the STARE framework schedules the perception model to operate on the event stream continuously, called Continuous Sampling, avoiding idle time caused by waiting for subsequent frames. The perception model can instantly receive input right after generating the current bounding box, which leads to a higher frequency of outputs and maximally exploits the real-time capability of the perception system.

With the same algorithm for tracking and evaluation as in frame-based latency-aware evaluation, we can get

$$N_{\text{streaming}} \leq N_{\text{STARE}}, \tag{26}$$

with equality if and only if

$$\Delta t = \lambda \cdot \frac{1}{H}, \lambda \in \mathbb{Z}^+. \tag{27}$$

Moreover, in the STARE framework, it follows that

$$N_{\text{gt}} - 1 < N_{\text{STARE}}, \text{ if } \Delta t < \frac{1}{H}. \tag{28}$$

In the real-world online scenarios, both H and N_{gt} can be considered infinite, while Δt cannot be equal to zero. However, in computer-simulated evaluations, they are all finite. Thus, a higher value of H and N_{gt} aids in a more precise evaluation of a perception model's real-time performance, to some extent motivating the creation of our ESOT500 dataset.

Reconstruction error

A significant secondary benefit of ESOT500's 500 Hz annotation rate is its ability to faithfully capture high-dynamic object motion without the temporal aliasing[34] inherent in lower-frequency datasets. Furthermore, the process of recording a continuous real-world motion trajectory at a discrete frequency can be viewed as a lossy information channel[45]. The original, continuous trajectory serves as the input signal, while the set of low-frequency annotations is the channel's output. The fundamental question is: how much information about the original signal is preserved after passing through this channel?

To quantify this, we define the Reconstruction Error (RE), which serves as a practical measure of the channel's fidelity. The RE evaluates our ability to reconstruct the original high-frequency trajectory $\mathcal{B}_{\text{high}}$, given only a downsampled set of low-frequency annotations $\mathcal{B}_{\text{low}}$. Specifically, for a given low frequency f_{low}, we generate $\mathcal{B}_{\text{low}}$ by subsampling $\mathcal{B}_{\text{high}}$. We then use a linear interpolator $\mathcal{L}$ to reconstruct the trajectory between every two consecutive low-frequency samples $\mathbf{B}_{t'_i}$ and $\mathbf{B}_{t'_{i+1}}$ in $\mathcal{B}_{\text{low}}$.

The RE at frequency f_{low} is then the accumulated loss between the reconstructed trajectory $\mathcal{B}_{\text{recon}}$ and the original high-frequency ground truth:

$$\text{RE}(f_{\text{low}}) = \mathbb{E}_{t \in [t'_i, t'_{i+1}]} \left[\text{Loss}(\mathbf{B}_t, \mathcal{L}((\mathbf{B}_{t'_i}, \mathbf{B}_{t'_{i+1}}), t)) \right], \tag{29}$$

where $\text{Loss}(\cdot, \cdot)$ is a metric such as 1 − IoU. Here we adopt L2 distance. A high reconstruction error signifies that the sampling frequency is too low to capture the signal's complexity, leading to severe information loss, which is also known as temporal aliasing[34]. This error is thus a direct, quantitative measure of aliasing, reflecting the gap between the information content of the original signal.

Details and insights of model enhancement

Predictive motion extrapolation. We define the function κ to represent the linear compensation function based on model-predicted velocity, which can be formally described as

$$\kappa(\hat{\mathbf{B}}, \hat{\mathbf{v}}, \Delta t) = \hat{\mathbf{B}} + \hat{\mathbf{v}} \cdot \Delta t, \tag{30}$$

where the latency Δt is a scalar, and $\hat{\mathbf{B}}$ and $\hat{\mathbf{v}}$ are both four-dimensional vectors, including the information of coordinates of the top-left corner and the size of the bounding box.

The predictive head is trained by adding the loss of predictive inference result $\hat{\mathbf{B}}_{\text{pred}}$, which is obtained by Eq. (30), in addition to the loss of inference result of current input, $\hat{\mathbf{B}}_{\text{curr}}$. The improved loss formula can be expressed as

$$\text{Loss}(\mathbf{B}_{\text{curr}}, \hat{\mathbf{B}}_{\text{curr}}) + \text{Loss}(\mathbf{B}_{\text{pred}}, \hat{\mathbf{B}}_{\text{pred}}), \tag{31}$$

where

$$\hat{\mathbf{B}}_{\text{pred}} = \kappa(\hat{\mathbf{B}}_{\text{curr}}, \hat{\mathbf{v}}_{\text{curr}}, \text{latency}). \tag{32}$$

The loss of $\hat{\mathbf{B}}_{\text{pred}}$ and $\hat{\mathbf{B}}_{\text{curr}}$ follows the same calculation formula defined by the specific perception model. Furthermore, considering the tracked object does not always move in a straight line, the parameter latency is set to a small value (e.g., 8 ms) to facilitate model convergence. In fact, the predictive module is more effective at predicting the short future, and when the required compensation time is too long, it can lead to incorrect predictions.

An intriguing finding during debugging was that simply equipping the model with predictive capability, even when its velocity prediction is not used for the final output, already boosts STARE performance. If we further use κ to adjust $\hat{\mathbf{B}}_{\text{curr}}$ according to $\hat{\mathbf{v}}_{\text{curr}}$ and Δt, the performance can be improved further.

Asynchronous tracking. The sequential perception pipeline described in Algo. 1 of Supplementary Note 5 inherently suffers from processing latency. During the inference time Δt of the perception model, any newly arriving events are queued and not immediately processed. This creates a temporal gap between the latest available sensory information and the model output, thus hindering true low-latency perception.

To mitigate this, we introduce an asynchronous tracking paradigm inspired by recent advancements in event-based vision[9]. As shown in Fig. 6a, this approach decouples the main tracking process into a heavyweight base model and a lightweight residual model. The base model is responsible for comprehensive feature extraction and state estimation, while the residual model provides rapid, incremental updates during the base model's inference period.

Let the base model $\mathcal{G}_{\text{base}}$ be composed of a feature extracting backbone $\mathcal{G}_b$ and a prediction head $\mathcal{G}_h$. In a standard cycle, the backbone first extracts a feature representation $\mathbf{h}_{\text{curr}}$ from the input event frame $\hat{\mathcal{E}}_{\text{curr}}$ with the last output bounding box $\hat{\mathbf{B}}_{\text{last}}$:

$$\mathbf{h}_{\text{curr}} = \mathcal{G}_b(\hat{\mathcal{E}}_{\text{curr}}, \hat{\mathbf{B}}_{\text{last}}). \tag{33}$$

The prediction head then computes the bounding box $\hat{\mathbf{B}}_{\text{base}}$ from this feature, a process which contributes to the overall latency Δt:

$$\hat{\mathbf{B}}_{\text{base}} = \mathcal{G}_h(\mathbf{h}_{\text{curr}}). \tag{34}$$

While the base model is computing $\mathbf{h}_{\text{curr}}$ and $\hat{\mathbf{B}}_{\text{base}}$, a new stream of events arrives. We term the latest events in this new stream, within the current sampling window, as the residual event segment $\mathcal{E}_{\text{res}}$. The residual model $\mathcal{G}_{\text{res}}$ is a lightweight network designed to process these new events quickly. It takes the latest backbone feature $\mathbf{h}_{\text{curr}}$ from the base model and the new residual event frame $\hat{\mathcal{E}}_{\text{res}} = \chi(\mathcal{E}_{\text{res}})$ as input to generate an updated feature map $\mathbf{h}_{\text{updated}}$:

$$\mathbf{h}_{\text{updated}} = \mathcal{G}_{\text{res}}(\mathbf{h}_{\text{curr}}, \hat{\mathcal{E}}_{\text{res}}). \tag{35}$$

This updated feature, which now incorporates the most recent motion information, is immediately fed into the base model's prediction head $\mathcal{G}_h$ to yield a refined, lower-latency bounding box $\hat{\mathbf{B}}_{\text{async}}$:

$$\hat{\mathbf{B}}_{\text{async}} = \mathcal{G}_h(\mathbf{h}_{\text{updated}}). \tag{36}$$

Notably, we schedule the residual model based on the speed ratio between the base model and the residual model running in parallel. For example, we might execute the residual model three times within the processing interval of a single base model run to ensure the speed stability of the parallel running. By generating $\widehat{\mathbf{B}}_{\text{async}}$ during the inference period, this asynchronous approach effectively utilizes the computational downtime of the sequential perception loop, leading to a more timely and accurate perception of the object state.

Context-aware sampling

Event density calculation. Following the given notations, we give a formal description of the function ϕ to calculate the event density ρ with respect to the bounding box area $\mathbf{B}_{-}$ of the last active output, which is defined as

$$\rho = \phi(\mathbf{B}_{-}, \mathcal{E}_{l,r}) = \sum_{\mathbf{x}\in\mathbf{B}_{-}} \psi(\mathbf{x}, \mathcal{E}_{l,r}), \tag{37}$$

and

$$\psi(\mathbf{x}, \mathcal{E}_{l,r}) = \begin{cases} 1, & \text{if } \exists(\mathbf{x}_i, t_i, p_i) \in \mathcal{E}_{l,r}, \text{ s.t. } \mathbf{x} = \mathbf{x}_i \\ 0, & \text{otherwise} \end{cases}. \tag{38}$$

The calculated density represents the number of event-triggered pixels within $\mathbf{B}_{-}$, instead of the full event number within $\mathbf{B}_{-}$. This is because, for perception models, whether there is an event triggered at a certain pixel position is far more critical than the total number of events triggered at that position.

Furthermore, for Eq. (37), it should be noted that since the reference event density is constantly updated and the size of $\mathbf{B}_{-}$ has significant uncertainty, it is advisable not to divide the area of $\mathbf{B}_{-}$ when calculating ρ.

Activated sampling judgement. With given notations, we give here a formal description of how we implement the active sampling judgement function $\mathcal{J}$, as

$$\mathcal{J}(\rho, \rho_{-}, d) = \begin{cases} \text{true}, & \text{if } \rho > \alpha \cdot \rho_{-} \\ \text{true}, & \text{if } \rho > \beta \cdot \rho_{-} \text{ and } d > \gamma \\ \text{false}, & \text{otherwise} \end{cases} \tag{39}$$

where ρ and ρ_{-} represent the event density of current sampling and last active sampling; d represent the lasting time of being inactive; α and β represent the high and low threshold ratios; γ represents the duration required to trigger the low threshold β. In our experiments, α and β are typically set to 0.5 and 0.05 respectively, and γ is set to 50ms.

As detailed in Section "Strategies for Continuous Event-driven Perception", every inactive sampling gives additional event data to the next cycle of tracking, enhancing its accuracy. However, the state of being inactive should not last for too long period, unless the object is really motionless, so we set an upper bound γ.

Triggering of the strategy. Given that some perception models rely on previous outputs for prediction assistance (e.g., tracking algorithms require the last bounding box to define the search area), Context-Aware Sampling can enhance prediction results even if it is not frequently triggered in a sequence. We present the trigger ratio (or sparsity) and the corresponding STARE performance improvement for a subset of ESOT500 sequences with the setting of 2 ms sampling window in Fig. 7c. More details are available in Supplementary Note 8.

Modified STARE pipeline. The Algo. 1 of Supplementary Note 5 illustrates the STARE pipeline. The modified pipeline, extended to Predictive Motion Extrapolation (blue) and Context-Aware Sampling (orange), can be seen in Algo. 2 of Supplementary Note 5. The modified pipeline for Asynchronous Tracking can be seen in Algo. 3 of Supplementary Note 5.

In the provided algorithm, it is evident that the Context-Aware Sampling serves as an enhancement that is independent of specific tracking algorithms. This strategy is designed to mitigate issues originating from the input data or from suboptimal parameter configurations. Conversely, the Predictive Motion Extrapolation represents a form of enhancement that inherently depends on the neural network structure of the perception model, and its implementation approach and associated challenges vary across different kinds of perception models.

Related work

We provide an overview of related work.

Asynchronous perception. To handle the unpredictability of high-dynamic motion where simple extrapolation can fail, we proposed Asynchronous Tracking. The most closely related work is from ref. 9, which builds a dual-component object detector with a high-precision, slow CNN and a lightweight, fast GNN to balance accuracy and speed. This work focuses on the trade-off between data bandwidth and computational efficiency (FLOPS) in object detection. However, the proposed slow model still relies on the RGB modality, and this dependency bottlenecks the speed at which features are updated and shared to the fast model. Our Asynchronous Tracking, however, is focused on object tracking and is driven by the pure event stream. The essential challenge of high-dynamic scenarios is their unpredictability[56,57]. This challenge should be addressed not by extrapolating further into an uncertain future, but by increasing the sampling and output frequency to minimize the accuracy degradation caused by perception latency.

Context-aware sampling. We designed context-aware sampling to address the sensitivity of event-based trackers to the sampling window size, as evidenced by the unimodal distributions observed in our experiments. In the neuromorphic vision community, related ideas for dynamically processing event streams exist. Existing approaches typically act as open-loop, feed-forward filters. These include methods like adaptive temporal windowing[58,59], fixed-count event packeting[22], and saliency-based spatial filtering[60,61]. These methods rely on low-level statistics to decide which events to feed into the model. In contrast, our Context-Aware Sampling operates in a closed-loop manner. Its sampling strategy is guided not only by the event stream statistics, but also by the model's previous output, specifically the event density within the last predicted bounding box. This creates a runtime feedback loop, allowing the model to actively regulate its own input based on its perceived state. This methodological shift from an open-loop filter to a closed-loop system is the fundamental difference.

Predictive motion extrapolation. Our Predictive Motion Extrapolation method was designed to explicitly compensate for perception latency by forecasting the object's future state. In the neuromorphic vision community, several ideas have been explored in motion compensation or prediction, often by adapting classical state estimation algorithms. For instance, several studies have employed Kalman Filters to process the asynchronous event stream, predicting the future trajectory of objects like moving hands or drones[25,62–65]. Others have utilized variations like particle filters[66,67] or assumed simpler constant-velocity models to achieve similar predictive capabilities[68,69]. More recent bio-inspired works have explored this problem by using an unsupervised SNN to learn motion features and predict ball trajectories[70]. While these classical and SNN-based methods rely on either handcrafted motion models or specialized, spike-based learning rules, our Predictive Motion Extrapolation is an end-to-end learning

system built upon a conventional Artificial Neural Network (ANN). The velocity vector used for extrapolation is not assumed or learned through unsupervised plasticity, but is directly learned as part of the perception model's supervised training, allowing it to capture more complex dynamics from the data.

Data availability

The ESOT500 datasets generated in this study have been deposited in the Hugging Face repository and are openly available at https://huggingface.co/datasets/sii-geai-lab/ESOT500. The VisEvent dataset used in this study is available via Dropbox at https://www.dropbox.com/scl/fo/r406wsgll56fy0hhhwu62/AFo3cjXjSI4Dzjn5nlnXNW0?rlkey=ecgyd26j1ycfl1jbm4pwc3vbn&e=2&st=rzf95buf&dl=0. The FE108 dataset, used in this study, is available under restricted access by the third-party owners. Access can be obtained upon request to the owners via their website at http://fe108.dluticcd.com. Source data are provided with this paper.

Code availability

The source code, data details, and pre-trained models for this study are provided as Supplementary Code 1 and are also available at https://github.com/ispc-lab/STARE.

References

1. Gollisch, T. & Meister, M. Eye smarter than scientists believed: neural computations in circuits of the retina. *Neuron* **65**, 150–64 (2010).
2. Wurtz, R. Neuronal mechanisms of visual stability. *Vis. Res.* **48**, 2070–89 (2008).
3. Gallego, G. et al. Event-based vision: a survey. *IEEE Trans. Pattern Anal. Mach. Intell.* **44**, 154–180 (2022).
4. Zhang, J. et al. Spiking transformers for event-based single object tracking. *2022 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)* 8791–8800 (2022).
5. Wang, X. et al. Event stream-based visual object tracking: a high-resolution benchmark dataset and a novel baseline. *2024 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)* 19248–19257 (2024).
6. Gehrig, M, Scaramuzza, D. Recurrent vision transformers for object detection with event cameras. *2023 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)* 13884–13893 (2023).
7. Klenk, S. et al. Deep event visual odometry. *2024 International Conference on 3D Vision (3DV)* 739–749 (2024).
8. Mueggler, E. et al. The event-camera dataset and simulator. *Int. J. Rob. Res.* **36**, 142–149 (2017).
9. Gehrig, D. & Scaramuzza, D. Low-latency automotive vision with event cameras. *Nature* **629**, 1034–1040 (2024).
10. Li, M, Wang, YX, Ramanan, D. *Towards streaming perception in Computer Vision - ECCV 2020* (eds Vedaldi, A., Bischof, H., Brox, T., Frahm, J. M.) (Springer International Publishing, Cham, 2020) 473–488.
11. Wang, X. et al. Are we ready for vision-centric driving streaming perception? the asap benchmark. *2023 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)* 9600–9610 (2023).
12. Yang, J. et al. Real-time object detection for streaming perception. *2022 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)* 5375–5385 (2022).
13. Li, B. et al. Pvt++: a simple end-to-end latency-aware visual tracking framework. *2023 IEEE/CVF International Conference on Computer Vision (ICCV)* 9972–9982 (2023).
14. Wang, Z. et al. Ev-catcher: high-speed object catching using low-latency event-based neural networks. *IEEE Robot. Autom. Lett.* **7**, 8737–8744 (2022).
15. Falanga, D., Kim, S. & Scaramuzza, D. How fast is too fast? the role of perception latency in high-speed sense and avoid. *IEEE Robot. Autom. Lett.* **4**, 1884–1891 (2019).
16. Falanga, D., Kleber, K. & Scaramuzza, D. Dynamic obstacle avoidance for quadrotors with event cameras. *Sci. Robot.* **5**, eaaz9712 (2020).
17. He, T. et al. Agile but safe: learning collision-free high-speed legged locomotion. *Proceedings of Robotics: Science and Systems* (2024).
18. Ramesh, B. et al. Long-term object tracking with a moving event camera. *British Machine Vision Conference* (2018).
19. Chen, H. et al. Asynchronous tracking-by-detection on adaptive time surfaces for event-based object tracking. *Proceedings of the 27th ACM International Conference on Multimedia*, MM '19 473-481 (2019).
20. Chen, H. et al. End-to-end learning of object motion estimation from retinal events for event-based object tracking. *Proc. AAAI Conf. Artif. Intell.* **34**, 10534–10541 (2020).
21. Zhang, J. et al. Object tracking by jointly exploiting frame and event domain. *2021 IEEE/CVF International Conference on Computer Vision (ICCV)* 13023–13032 (2021).
22. Mostafavi, M., Wang, L., Yoon, K. J. Learning to reconstruct hdr images from events, with applications to depth and flow prediction. *Int. J. Comput. Vis.* **129**, 900–920 (2021).
23. Gehrig, D. et al. End-to-end learning of representations for asynchronous event-based data. *2019 IEEE/CVF International Conference on Computer Vision (ICCV)* 5632–5642 (2019).
24. Wang, X. et al. Visevent: reliable object tracking via collaboration of frame and event flows. *IEEE Trans. Cybern.* **54**, 1997–2010 (2024).
25. Li, Z. et al. Hybrid object tracking with events and frames. *2023 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)* 9057–9064 (2023).
26. Kang, Y. et al. Direct 3d model-based object tracking with event camera by motion interpolation. *2024 IEEE International Conference on Robotics and Automation (ICRA)* 2645-2651 (2024).
27. Glover, A. et al. Edopt: Event-camera 6-dof dynamic object pose tracking. 2024 IEEE International Conference on Robotics and Automation (ICRA) 18200–18206 (2024).
28. Ahmed, S. H., Finkbeiner, J., Neftci, E. Efficient event-based object detection: a hybrid neural network with spatial and temporal attention. *2025 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)* 13970–13979 (2025).
29. Zhu, Z., Hou, J. & Lyu, X. Learning graph-embedded key-event back-tracing for object tracking in event clouds. *Adv. Neural Inf. Process. Syst.* **35**, 7462–7476 (2022).
30. Dampfhoffer, M. et al. Graph neural network combining event stream and periodic aggregation for low-latency event-based vision. *2025 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)* 6909–6918 (2025).
31. Annamalai, L., Ramanathan, V. & Thakur, C. S. Event-lstm: An unsupervised and asynchronous learning-based representation for event-based data. *IEEE Robot. Autom. Lett.* **7**, 4678–4685 (2022).
32. Sun, T. & Bohté, S. Dpsnn: spiking neural network for low-latency streaming speech enhancement. *Neuromorphic Comput. Eng.* **4**, 044008 (2024).
33. Chu, J. et al. The ESOT500 dataset for high-dynamic event-driven visual object tracking. *Hugging Face* https://huggingface.co/datasets/sii-geai-lab/ESOT500 (2025).
34. Nyquist, H. Certain topics in telegraph transmission theory. *Trans. Am. Inst. Electr. Eng.* **47**, 617–644 (1928).
35. Li, B. et al. High performance visual tracking with siamese region proposal network. *2018 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)* 8971-8980 (2018).
36. Jung, I. et al. *Real-Time mdnet in Computer Vision - ECCV 2018* (eds Ferrari, V., Hebert, M., Sminchisescu, C., Weiss, Y.) 89–104 (Springer International Publishing, Cham, 2018).

37. Yan, B. et al. Learning spatio-temporal transformer for visual tracking. *2021 IEEE/CVF International Conference on Computer Vision (ICCV)* 10428-10437 (2021).
38. Chen, X. et al. Transformer tracking. *2021 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)* 8122–8131 (2021).
39. Yan, B. et al. Alpha-refine: Boosting tracking performance by precise bounding box estimation. *2021 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)* 5285–5294 (2021).
40. Kim, S., Shukla, A. & Billard, A. Catching objects in flight. *IEEE Trans. Robot.* **30**, 1049–1065 (2014).
41. DAmbrosio, D. B. et al. Achieving human level competitive robot table tennis. *2025 IEEE International Conference on Robotics and Automation (ICRA)* 74–82 (2025).
42. Salehian, S. S. M., Khoramshahi, M. & Billard, A. A dynamical system approach for softly catching a flying object: theory and experiment. *IEEE Trans. Robot.* **32**, 462–471 (2016).
43. Sato, M., Takahashi, A., Namiki, A. High-speed catching by multi-vision robot hand. *2020 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)* 9131–9136 (2020).
44. Ma, Y., Cramariuc, A., Farshidian, F. & Hutter, M. Learning coordinated badminton skills for legged manipulators. *Sci. Robot.* **10**, eadu3922 (2025).
45. Shannon, C. Communication in the presence of noise. *Proc. IRE* **37**, 10–21 (1949).
46. Danelljan, M., Van Gool, L., Timofte, R. Probabilistic regression for visual tracking. *2020 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)* 7181–7190 (2020).
47. Cui, Y., Jiang, C., Wang, L., Wu, G. Mixformer: End-to-end tracking with iterative mixed attention. *2022 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)* 13598–13608 (2022).
48. Mayer, C. et al. Learning target candidate association to keep track of what not to track. *2021 IEEE/CVF International Conference on Computer Vision (ICCV)* 13424–13434 (2021).
49. Bhat, G. et al. Know your surroundings: Exploiting scene information for object tracking. in *Computer Vision - ECCV 2020: 16th European Conference*, Glasgow, UK, August 23-28, 2020, Proceedings, Part XXIII 205-221 (Springer-Verlag, Berlin, Heidelberg, 2020).
50. Paul, M. et al. Robust visual tracking by segmentation. in *Computer Vision - ECCV 2022: 17th European Conference, Tel Aviv, Israel, October 23-27, 2022, Proceedings, Part XXII* 571–588 (Springer-Verlag, Berlin, Heidelberg, 2022).
51. Wang, S. et al. Mamba-fetrack v2: Revisiting state space model for frame-event based visual object tracking. *arXiv* (2025).
52. Wu, H. A comprehensive of cpu and gpu performance and applications in autonomous vehicles. *Sci. Technol. Eng. Chem. Environ. Prot.* **1**, 1–6 (2024).
53. NVIDIA Corporation. The Ultimate Platform for Physical AI and Robotics. *NVIDIA* https://www.nvidia.com/en-us/autonomous-machines/embedded-systems (2024).
54. Ghosh, A. et al. Chanakya: Learning runtime decisions for adaptive real-time perception. in *Advances in Neural Information Processing Systems* (eds Oh, A., Naumann, T., Globerson, A., Saenko, K., Hardt, M., Levine, S.) Vol. 36, 55668–55680 (Curran Associates, Inc., 2023).
55. Wu, Q. et al. *Directly Forecasting Belief for Reinforcement Learning with Delays* (PMLR, 2025).
56. Cover, T. M. *Elements of Information Theory* (John Wiley & Sons, 1999).
57. Ay, N. et al. Predictive information and explorative behavior of autonomous robots. *Eur. Phys. J. B* **63**, 329–339 (2008).
58. Ye, C. et al. Unsupervised learning of dense optical flow, depth and egomotion with event-based sensors. *2020 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)* 5831–5838 (2020).
59. Stoffregen, T., Kleeman, L. Event cameras, contrast maximization and reward functions: an analysis. *2019 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)* 12292–12300 (2019).
60. Lagorce, X. et al. Hots: a hierarchy of event-based time-surfaces for pattern recognition. *IEEE Trans. Pattern Anal. Mach. Intell.* **39**, 1346–1359 (2017).
61. Asgari, H., Risi, N., Indiveri, G. Fpga implementation of an event-driven saliency-based selective attention model. *2022 IEEE Biomedical Circuits and Systems Conference (BioCAS)* 307–311 (2022).
62. Wang, Z. et al. Asynchronous blob tracker for event cameras. *IEEE Trans. Robot.* **40**, 4750–4767 (2024).
63. Mitrokhin, A. et al. Event-based moving object detection and tracking. *2018 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)* (2018).
64. Rudnev, V. et al. Eventhands: Real-time neural 3d hand pose estimation from an event stream. *2021 IEEE/CVF International Conference on Computer Vision (ICCV)* 12365–12375 (2021).
65. Theurkauf, A. et al. Chance-constrained motion planning with event-triggered estimation. *2023 IEEE International Conference on Robotics and Automation (ICRA)* 7944–7950 (2023).
66. Duarte, L., Safeea, M. & Neto, P. Event-based tracking of human hands. *Sens. Rev.* **41**, 382–389 (2021).
67. Younis, A., Sudderth, E. B. Differentiable and stable long-range tracking of multiple posterior modes. *Thirty-Seventh Conference on Neural Information Processing Systems* (2023).
68. Niu, J. et al. Esvo2: Direct visual-inertial odometry with stereo event cameras. *IEEE Transactions on Robotics* PP 1-20 (2025).
69. Zhu, Q., Triesch, J. & Shi, B. E. An event-by-event approach for velocity estimation and object tracking with an active event camera. *IEEE J. Emerg. Sel. Top. Circuits Syst.* **10**, 557–566 (2020).
70. Debat, G. et al. Event-based trajectory prediction using spiking neural networks. *Front. Comput. Neurosci.* **15**, 658764 (2021).
71. Mitrokhin, A. et al. Ev-imo: Motion segmentation dataset and learning pipeline for event cameras. *2019 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)* 6105–6112 (2019).
72. Burner, L. et al. Evimo2: an event camera dataset for motion segmentation, optical flow, structure from motion, and visual inertial odometry in indoor scenes with monocular or stereo algorithms. *arXiv* (2022).
73. Tang, C. et al. Revisiting color-event based tracking: A unified network, dataset, and metric. *arXiv* (2024).
74. Zhu, Y. et al. Crsot: cross-resolution object tracking using unaligned frame and event cameras. *IEEE Trans. Multimed.* **27**, 6529–6542 (2025).
75. Wang, X. et al. Long-term visual object tracking with event cameras: An associative memory augmented tracker and a benchmark dataset. *arXiv* (2025).
76. Brandli, C. et al. A 240 × 180 130 db 3 μs latency global shutter spatiotemporal vision sensor. *IEEE J. Solid-State Circuits* **49**, 2333–2341 (2014).
77. Taverni, G. et al. Front and back illuminated dynamic and active pixel vision sensors comparison. *IEEE Trans. Circuits Syst. II Express Briefs* **65**, 677–681 (2018).
78. Prophesee. Prophesee gen3.1 vga sensor documentation. *Prophesee* https://docs.prophesee.ai/stable/hw/sensors/gen31.html (2025).
79. Posch, C., Matolin, D. & Wohlgenannt, R. A qvga 143 db dynamic range frame-free pwm image sensor with lossless pixel-level video compression and time-domain cds. *IEEE J. Solid-State Circuits* **46**, 259–275 (2011).
80. Son, B. et al. 4.1 a 640 × 480 dynamic vision sensor with a 9μm pixel and 300meps address-event representation. *2017 IEEE International Solid-State Circuits Conference (ISSCC)* 66–67 (2017).

81. Prophesee. Prophesee evk4-hd documentation. *Prophesee* https://www.prophesee.ai/event-camera-evk4/ (2025).
82. Finateu, T. et al. 5.10 a 1280 × 720 back-illuminated stacked temporal contrast event-based vision sensor with 4.86μm pixels, 1.066geps readout, programmable event-rate controller and compressive data-formatting pipeline. *2020 IEEE International Solid-State Circuits Conference - (ISSCC)* 112–114 (2020).
83. Chen, S., Guo, M. Live demonstration: Celex-v: A 1m pixel multimode event-based sensor. *2019 IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops (CVPRW)* 1682–1683 (2019).
84. DVSense. DVSync documentation. *DVSense* https://dvsense.com/dvsync (2025).
85. Ye, B. et al. Joint feature learning and relation modeling for tracking: a one-stream framework. in *Computer Vision - ECCV 2022* (eds Avidan, S., Brostow, G., Cissé, M., Farinella, G. M., Hassner, T.) 341–357 (Springer Nature Switzerland, Cham, 2022).

Acknowledgements

This work was supported by funding from the National Key Research and Development Program of China (No. 2024YFE0211000 [G.C.]), in part by the European Union's Horizon 2020 Framework Program for Research and Innovation under the Specific Grant Agreements No. 945539 [A.K.] (Human Brain Project SGA3), in part by the National Natural Science Foundation of China (No. 62372329 [G.C.]), in part by Shanghai Scientific Innovation Foundation (No. 23DZ1203400 [G.C.]), in part by Tongji-Qomolo Autonomous Driving Commercial Vehicle Joint Lab Project (No. 0170920220536 [G.C.]), and in part by Xiaomi Young Talents Program (No. 2023 [G.C.]).

Author contributions

G.C. conceived the main idea. G.C., A.K. and C.J. supervised the work. G.C., J.C. and R.Z. led the work. J.C., R.Z., C.Y., and Z.B. developed the code for the core algorithm. J.C., R.Z., C.Y., Z.Y. and Z.B. collected and verified the dataset. J.C., R.Z., C.Y., Z.Y. and Z.B. designed and performed the data analysis. J.C., R.Z. and C.Y. designed the experiments. J.C., R.Z. and Z.Y. designed and performed the real-robot experiments. J.C., R.Z. and C.Y. wrote the initial version of the manuscript. J.C., R.Z., Z.Y., H.L., F.R., A.K., G.C. and C.J. revised the manuscript.

Competing interests

The authors declare no competing interests.

Additional information

Supplementary information The online version contains supplementary material available at https://doi.org/10.1038/s41467-026-70240-6.

Correspondence and requests for materials should be addressed to Guang Chen.

Peer review information *Nature Communications* thanks the anonymous, reviewer(s) for their contribution to the peer review of this work. A peer review file is available.